PROJECT ROADMAP & ARCHITECTURE

แผนงานพัฒนาระบบเกมการ์ดออนไลน์ (PVP Online)

สรุปข้อกำหนดเชิงสถาปัตยกรรม (Authoritative Server), ตารางฟีเจอร์, และแผนการปรับปรุงระบบเดิม

Game Skeleton Step 4E2B
สถานะระบบ: 352/352 Tests ผ่าน 100%
วันที่จัดทำ: 28 ตุลาคม 2026

> 💡 **จุดแข็งเชิงสถาปัตยกรรมของเกมในปัจจุบัน:**
> โครงสร้างเกมถูกออกแบบตามมาตรฐานสากล โดยแยกตรรกะกติกาในโฟลเดอร์ `engine/` ออกจากฝั่งแสดงผล UI 100% และรับคำสั่งผ่านระบบ Action Object เช่น `applyAction(state, action)` ซึ่งเป็น Deterministic State Machine จึงสามารถนำ `engine/` ไปรันบน Server Node.js ได้ทันที โดยไม่ต้องเขียนลอจิกกฎกติกาการ์ดใหม่แม้แต่บรรทัดเดียว

📋 ตารางที่ 1: แผนงานพัฒนาระบบออนไลน์ (Master Online Roadmap)

เฟส	โมดูล / ระบบ (Module)	รายละเอียดหน้าที่และการทำงาน	ระดับความสำคัญ	เป้าหมายไฟล์
Phase 1 ต้นแบบออนไลน์	**1. WebSocket Server**	สร้าง Server ด้วย Node.js + WebSocket (`ws`) รันเป็นศูนย์กลางควบคุม State แบบ Real-time	จำเป็นขั้นวิกฤต	`server/server.js`
	2. Room System	ระบบสร้างห้องเล่น (Create Room) ได้รหัสห้อง 5 ตัวอักษร และระบบเข้าร่วมห้อง (Join Room)	จำเป็นขั้นวิกฤต	`server/rooms.js` `ui/app.js`
	3. Network Action Sync	ปรับ `ui/app.js` ให้ส่ง Action ผ่าน Socket เมื่อคลิก และรอรับ State ใหม่จาก Server มาวาดจอ	จำเป็นขั้นวิกฤต	`ui/network.js` `ui/app.js`
Phase 2 ความปลอดภัย	**4. Fog of War (ซ่อนข้อมูล)**	ลบการ์ดในมือศัตรู, กองเด็ค, และการ์ดคว่ำออกจาก State ก่อนส่งให้ Client เพื่อป้องกันการกด F12 แอบส่องไพ่	จำเป็นขั้นวิกฤต	`server/sanitize.js`
	5. Deck Validation	ตรวจสอบเด็คที่ส่งมาจากผู้เล่นบน Server ว่าถูกต้องตามกติกาหรือไม่ (ไม่ใส่การ์ดเกินจำนวน, ไม่มีรหัสการ์ดปลอม)	จำเป็นสูง	`engine/deck_builder.js`
	6. Action Authority Check	ตรวจเช็กบน Server ว่าผู้เล่นที่ส่งคำสั่งเข้ามาเป็นเจ้าของเทิร์นหรือสิทธิ์ตอบโต้จริง ป้องกันการส่งคำสั่งข้ามคิว	จำเป็นสูง	`server/server.js`
Phase 3 ประสบการณ์เล่น	**7. Turn & Chain Timers**	นาฬิกานับถอยหลังต่อเทิร์น (เช่น 60 วิ) และต่อการตัดสินใจ Chain (เช่น 15 วิ) หากหมดเวลาจะ Auto-Pass ป้องกันการดองเกม	จำเป็นสูง	`server/timer.js` `ui/view.js`
	8. Fast Priority Pass	สวิตช์ตั้งค่าตอบโต้ (AUTO / ON / OFF) หากไม่มีการ์ดที่ใช้งานได้ ให้ข้ามสิทธิ์ตอบโต้ทันที ไม่ทำให้เกมสะดุด	แนะนำ	`ui/app.js` `engine/actions.js`
	9. Disconnect & Reconnect	ให้เวลาผู้เล่นเน็ตหลุด 45 วินาทีในการเชื่อมต่อกลับมาดวลต่อ หากเกินกำหนดปรับแพ้ (Surrender)	แนะนำ	`server/rooms.js`
Phase 4 สมบูรณ์แบบ	**10. Event Animation Queue**	เล่นแอนิเมชันทีละขั้น (ร่ายการ์ด -> โจมตี -> การ์ดถูกทำลาย) เพื่อให้ผู้เล่นอีกฝั่งเห็นภาพชัดเจนและเข้าใจเหตุการณ์	แนะนำ	`ui/app.js` `ui/style.css`
	11. Cloud Deployment	ติดตั้ง Server บนบริการคลาวด์ เช่น Render, Railway หรือ VPS เพื่อให้เพื่อนจากต่างบ้าน	ส่วนขยาย	`Dockerfile / Cloud`

เฟส	โมดูล / ระบบ (Module)	รายละเอียดหน้าที่และการทำงาน	ระดับความสำคัญ	เป้าหมายไฟล์
		เชื่อมต่อเล่นได้		

REFACTORING & EXECUTION

การปรับปรุงระบบเดิม และแนวทางปฏิบัติงาน

รายการระบบที่ต้องปรับปรุง, ระบบป้องกันการโกง และเช็กลิสต์เริ่มต้น

🔧 ตารางที่ 2: สิ่งที่ควรปรับปรุงจากระบบปัจจุบัน (Refactoring & UX Polish)

หัวข้อการปรับปรุง	สภาพปัจจุบัน (Current State)	แนวทางแก้ไขและปรับปรุงใหม่ (Target Improvement)
1. จังหวะการเล่น (Visual Pacing)	การเปลี่ยนสถานะเกิดขึ้นทันทีในพริบตา (Instant Render)	สร้างคิวแอนิเมชันให้การ์ดลอยหรือมีเอฟเฟกต์แสดงชัดเจน เพื่อให้อีกฝ่ายตามทันว่าเกิดอะไรขึ้น
2. ระบบเสียง (Sound Effects)	ปัจจุบันไม่มีเสียงประกอบใดๆ ในเกม	เพิ่มเสียงวางการ์ด, เสียงจั่ว, เสียงโจมตี, เสียง LP ลดลง, และเพลง BGM บรรยากาศการดวล
3. บันทึกการเล่น (Battle Log)	เป็นแถบข้อความตัวหนังสือล้วนในกล่องด้านขวา	ใส่รูปไอคอนการ์ดเล็กๆ และสามารถคลิกที่ Log เพื่อดูรายละเอียดการ์ดที่เพิ่งเล่นไปได้
4. เมนูล็อบบี้ (Lobby Modes)	มีเฉพาะโหมด 'ซ้อมกับบอท' และ '2 ผู้เล่นส่งเครื่องกัน'	เพิ่มตัวเลือก '🌐 เล่นออนไลน์ (Online PVP)' พร้อมช่องกรอกชื่อผู้เล่นและระบบห้อง
5. ขนาดจอมือถือ (Responsive)	หน้าจอออกแบบสัดส่วนตามหน้าจอคอมพิวเตอร์	ปรับ Touch Event และสัดส่วนสนามให้รองรับการหมุนจอแนวนอนบนสมาร์ตโฟน/แท็บเล็ต

🛡️ ตารางที่ 3: ระบบความปลอดภัยและการป้องกันการโกง (Anti-Cheat Specification)

ช่องโหว่ที่อาจเกิดขึ้น	ความเสี่ยง (Risk Impact)	กลไกป้องกันทางเทคนิค (Prevention Mechanism)
แอบส่องไพ่บนมือ & ในเด็ค	ผู้เล่นเปิด Console ดูการ์ดที่ศัตรูถืออยู่ล่วงหน้า	ฟังก์ชัน `sanitizeState` จะลบ ID การ์ดทั้งหมดออก ฝั่งรับจะเห็นเพียง `{ count: 5 }`
แอบดูการ์ดที่คว่ำอยู่	รู้ว่าอีกฝ่ายหมอบกับดักหรือมอนสเตอร์อะไรไว้	Server ส่งเฉพาะ `{ faceDown: true, position: 'def' }` ไม่ส่งค่าพลังหรือชื่อการ์ด
ส่ง Action ปลอมข้ามเทิร์น	สั่งโจมตีทั้งที่ไม่ใช่เทิร์นตนเอง หรือสั่งทิ้งการ์ดศัตรู	Server ตรวจสอบด้วย `E.getLegalActions()` หากไม่อยู่ในรายการที่อนุญาต จะปฏิเสธทันที
ดองเกม / ไม่ยอมกดจบเทิร์น	จงใจทำให้ผู้เล่นอีกคนเสียเวลาจนต้องกดยอมแพ้	Server จับเวลาต่อเทิร์น หากหมดเวลาจะสั่งการกระทำ `endTurn` ให้โดยอัตโนมัติ

🚀 แผน 3 ขั้นแรกเมื่อพร้อมเริ่มลุยงาน (Quick-Start Checklist):

1. **สร้าง Server ต้นแบบ (Minimal Server):** เขียนไฟล์ `server/server.js` ขนาดกะทัดรัดด้วยแพ็กเกจ `ws` เพื่อทดสอบสร้างห้องและเชื่อมต่อ 2 เบราว์เซอร์ในเครื่องเดียวกันหรือในวง Wi-Fi เดียวกัน
2. **เชื่อมโยง UI เข้ากับ Socket:** นำ Action จากการคลิกใน `ui/app.js` ยิงผ่าน Socket ไปยัง Server และอัปเดต State พร้อมกัน 2 หน้าจอ
3. **ใส่ระบบ Fog of War:** ซ่อนการ์ดคว่ำและการ์ดบนมืออีกฝ่ายให้แสดงเป็นหลังการ์ดที่ถูกต้อง